Volt & Co — four demonstrations, one improving service

Presenter-only delivery contract

90 minutes for approximately 50 technical attendees: 80 minutes content, then 10 minutes Q&A. Use four sustained demonstrations instead of switching windows after every feature. All slide numbers below refer to the current 23-slide deck: 21 visible, two hidden backups.

The switch map: slides 1–8 → **slide 9 / Demo 1** → **slide 10**; slides 10–12 → **slide 13 / Demo 2** → **slide 14**; slides 14–15 → **slide 16 / Demo 3** → **slide 17**; slides 17–19 → **slide 20 / Demo 4** → **slide 21**. Demo cards visibly say where to switch and which slide to return to. Return slides carry a matching badge.

How to operate: show the demo card, say its opening bridge, then Alt+Tab to the prepared application group. Stay out of PowerPoint until the demo's return cue. Alt+Tab back, advance once to the numbered return slide, and say its debrief before introducing the next question. The deck does not launch commands automatically. Do not advance through conceptual slides while the audience should be watching a demonstration.

Room setup: use Presenter View on the private display. Use at least 22–26-point editor/terminal text and 125–150% browser zoom, rehearsed on the actual projector. Close unrelated tabs and notifications. Use only a few selected code lines; never project an entire JSON blob, credentials, connection metadata or private Memory identifiers.

Live versus prepared: Demo 1 and the end of Demo 4 send genuinely new requests to DEMO v1. Configuration is inspected in the live portals. Native evaluations, Optimizer, the already-merged issue/PR and previous guardrail/Memory proofs are completed evidence, not jobs secretly recomputed on stage. The offline viewer is always announced as **RECORDED EVIDENCE — NOT LIVE**. A pending or failed request is never narrated as a pass.

Safety and evidence boundaries: all customers, products, orders and documents are synthetic. Tools are read-only: no ticket, refund, shipment or handoff is executed. Memory is one-caller DEV proof, not verified cross-user isolation, and stays off in DEMO. Optimizer retained its baseline; no improved candidate was applied. Business scores are 8/9 → 9/9 → independent DEMO 9/9, not availability or universal correctness. The original incident has native response/evaluation evidence but no verified exported App Insights span; the separate golden DEMO request has 43 correlated spans.

Readiness: the technical evidence exists; a full human 90-minute timing/navigation rehearsal is still required. Use `docs/OPERATIONS.md` for setup and `Rehearsal bindings.json` for actual version/run links. The reorganization did not redeploy the agent or rebuild its verified release ZIP. Do not provision, launch a new Optimizer job, merge or promote on stage.

Slide 01 — From useful agent to governed service

Clock 00:00–01:00 · 1 minute slides · Screen: wave cover.

[Start on the cover. Do not open the terminal.]

“Welcome. Today we are going to follow one customer and one agent all the way from a useful answer to a service we can govern and improve.

“Our customer bought a Volt charger. Their request sounds simple: can I get a replacement? But answering responsibly requires product facts, policy evidence, permitted tools, a clear safety boundary and a way to know whether a change really improved the result.

“Microsoft Foundry is the agent platform in this story. Citadel gives us a governance architecture around it. GitHub Copilot helps us develop and repair the application; it is not the application runtime we are about to inspect.

“We have eighty minutes for the journey and ten for questions. Most of the time will be in the application and the platform, not in slides. Everything customer-facing is synthetic. When we inspect a prepared run, I will say so.

“The organizing picture is two loops.”

[At 01:00, advance to slide 2. If the audience arrives late, do not restart this introduction; recover time from optional inspection depth.]

Slide 02 — Two loops. One improving service.

Clock 01:00–04:00 · 3 minutes slides · Screen: the adapted original workshop inner/outer-loop diagram.

[Point to the left loop, then the right loop. Keep the actual diagram visible throughout.]

“This is the workshop's two-loop picture, adapted to the environments we are using today. The inner loop is where we build and prove the agent. We connect a model, tools and knowledge; run a request; inspect a trace; evaluate behavior; and improve the configuration.

“The outer loop is where we make that learning repeatable. A reviewed candidate passes release checks, reaches the serving environment and is observed in operation. We call that environment DEMO because it is synthetic and production-like, not customer production.

“The important parts are the two bridges. Look at the top bridge. We do not hand over a promising chat transcript and say, ‘ship it.’ We hand over an identifiable candidate and its evidence: code, instructions, tool configuration, knowledge, evaluation results and policy.

“Now look at the bottom bridge. Operational evidence comes back into development. That does not mean every answer the agent produced becomes training data or ground truth. A person reviews what happened and writes the expected behavior before a trace becomes a regression case.

“The center column is the shared contract. Environments can separate identities and permissions while sharing the definition of correct behavior. Our DEV and DEMO projects currently share an account, and the Memory identity's role scope is broader than a project. This profile is not a hard production isolation boundary.

“Here is a useful diagnostic question for your own projects. If someone changes a prompt tomorrow, can you tell which tools and policy were used, which tests passed, and which version is serving? If the answer is no, the missing piece is not necessarily a smarter model. It may be one of these bridges.

“We will travel clockwise. First we make the answer useful, then govern it, examine one narrow memory preference, measure it and inspect optimization. Finally, an observed decision mismatch becomes the evidence for a reviewed regression and a Copilot-assisted repair through the release gate.

“Keep this picture in mind. Each later screen is a close-up of one part of it.”

[Pause for five seconds to let the diagram settle. At 04:00 advance to slide 3. No live demo is needed here. If projection is too small to read the diagram, explain the two bridges rather than attempting to read every label.]

Slide 03 — One customer. One support journey.

Clock 04:00–06:00 · 2 minutes slides · Screen: customer case and three-step decision path.

“Meet our one customer. They have a Volt Charge 65, batch CH-0722, on order VC-10482. They are outside the standard thirty-day return window. They ask whether they can get a replacement and which policy applies.

“The temptation is to jump straight to a confident answer. We are going to resist that. A standard return window, a recall and a warranty or replacement assessment are different things. The model should not collapse them into one yes-or-no refund decision.

“First, business tools establish facts. What product and batch are involved? What does the synthetic recall record say? What is the order status, and what does the eligibility service return?

“Second, Foundry IQ supplies policy evidence. We want an identifiable policy source that we can open, not a plausible paragraph that the model invented.

“Third, the agent explains the responsible next step. For this journey, that means human assessment rather than an automatic refund. We are not moving money, sending replacement stock or changing an order on stage.

“The wider fictional store has catalog and compatibility capabilities as well, but we are not going on five separate tours. The replacement question is our thread through every platform feature.

“Later we will see why getting the facts and the prose right is still not enough. The structured workflow decision must agree with the tool's eligibility result. That will be our operational learning moment.

“Before we run the request, we need to choose who owns the agent's runtime.”

[At 06:00 advance to slide 4. If asked whether these are real customers, answer: “No. The product, order, recall and documents are synthetic fixtures, chosen so the expected behavior can be checked.”]

Slide 04 — Choose the operating model

Clock 06:00–08:00 · 2 minutes slides · Screen: prompt-versus-hosted comparison.

“There are two operating models worth separating. A prompt agent gives you the managed configuration path. You define instructions and capabilities without maintaining custom orchestration code. That is a good starting point when it meets the application's needs.

“A hosted agent gives you control of the application's runtime logic, while Foundry still supplies managed hosting. That is the choice for this demonstration because we want a deterministic decision adapter and the integration path we are inspecting.

“Our runtime is Python Microsoft Agent Framework, exposing the Responses protocol. Protocol choice matters here because the hosted optimization path we intend to demonstrate requires Responses. An earlier Copilot SDK Invocations implementation exists in Git history; it is not the runtime or a fallback for this workshop.

“Tools, source control and CI are not, on their own, reasons to choose hosted. The real question is whether you need the custom runtime behavior and are prepared to own that code and its dependencies.

“GitHub Copilot will appear later as our development assistant. It can help write a regression and repair the adapter. That does not make it the serving harness.

“There are also other managed-harness experiences in preview. They are useful context, but they are not another implementation track today. We will not turn this into a survey of every agent option.

“Notice the cost row. Hosting adds active compute to model and tool usage. We have not measured a total cost advantage for this application, so I am not going to promise one. The choice should follow control, compatibility and operating requirements.

“Let us make that ownership split concrete.”

[At 08:00 advance to slide 5. Keep questions about alternate SDKs for Q&A unless they block understanding of the chosen runtime.]

Slide 05 — Our code. Foundry's runtime.

Clock 08:00–10:00 · 2 minutes slides · Screen: responsibility split.

“Hosted is the control path, not the unmanaged path. On the left are our responsibilities: the agent logic, orchestration, framework dependencies, request handling and configuration. If our application's final decision contradicts its authoritative tool evidence, we have to investigate and repair that decision path. Managed hosting cannot repair the business meaning for us.

“On the right is the platform: managed deployment and endpoints, scaling and agent identity, and the lifecycle surfaces for tracing, evaluation and operation.

“The deployed agent can be tested and inspected in Foundry, but we do not edit the hosted Python implementation in the prompt-agent editor. We change code in source control and deploy a candidate.

“Responses is the contract at the hosted boundary. The runtime can reach capabilities through the project and Toolbox, and our application must still supply the correct configuration and authorization context.

“Here is the distinction to remember when Copilot appears. Copilot is helping the developer work on code. Agent Framework is running the customer request. If we confuse those two roles, we will misunderstand both the lifecycle and the evidence.

“Similarly, a managed identity does not mean every downstream hop is keyless. We will call out the actual access contracts at the gateway rather than generalize from one identity badge.

“Now we can put the agent into the broader Citadel picture.”

[At 10:00 advance to slide 6.]

Slide 06 — Where Citadel and Foundry meet

Clock 10:00–13:00 · 3 minutes slides · Screen: original Citadel architecture, then prepared workspace.

“This is the supplied Citadel architecture. We have kept the original layered picture because the relationships matter more than a list of products.

“Layer two is agent operations: build, host, evaluate and improve. That is where our Volt agent, its tools, context and evaluations live.

“Layer one is the Governance Hub. In our selected routes, API Management sits between the agent-facing connection and the approved model or business tools. It gives us an explicit place to apply access and consumption controls.

“The diagram also shows broader management and security integrations. Those are architecture context. Deploying a gateway does not automatically enable Defender, Agent 365, Purview or every other layer. We only claim a control after inspecting its actual configuration and behavior.

“There are also service paths not shown as a single universal gateway hop. We must not assume that IQ internals, memory processing, evaluation models or optimizer-service traffic all traverse our model gateway and inherit its quotas.

“Our account has separate DEV and DEMO projects, but those are not hard production isolation in this profile. The effective account-managed-identity role scope used for Memory is broader than project scope. Memory and evaluation model traffic are not automatically under the customer's APIM token budget.

“The deployed DEMO runtime is narrower: project-only Foundry User, no Memory account roles and Memory disabled. It has separate gateway contracts and connections. That improves this runtime's boundary without turning the shared account into a claim of hard production isolation.”

“The selected resource group is in Sweden Central, but resource location is not a Sweden-only processing guarantee. The deployed model uses a data-zone SKU. We need to discuss processing geography and retention separately from the map pin.

“The Foundry endpoint is public and Entra-authenticated in the approved configuration. Key Vault and backend storage remain private. This is not an all-private-network demonstration.

“With those boundaries clear, let us connect the tools and policy before our first live request.”

[Stay in the deck. At 13:00 advance to slide 7. The code peek moves into Demo 1; do not make a fifth, unannounced screen switch here.]

Slide 07 — Give the agent governed capabilities

Clock 13:00–14:00 · 1 minute slides.

“The Toolbox is how we package the capabilities the agent may use. For this request, the useful business operations are recall, order status and replacement eligibility. These are read-only operations, not a license to mutate an order.

“Read the diagram left to right. We expose a curated set, pin its version and bind the hosted agent to it. The backend may support more operations than this agent is allowed to see.

“In the first demo we will compare this configuration with what actually ran. A tool appearing in a catalog is not proof that the agent used it. First, let us add the policy evidence that makes the answer explainable.”

[Remain in PowerPoint. Point to tools, Toolbox and agent, then advance to slide 8. The visible Demo 1 card on slide 9 is the switch point.]

Slide 08 — Ground the answer in policy

Clock 14:00–15:00 · 1 minute slides.

“Business tools establish facts; policy retrieval explains what those facts mean. This is the source workshop's source-to-knowledge-to-agent pattern, narrowed to our actual Azure AI Search implementation.

“The knowledge base uses text and semantic extractive retrieval. I am not claiming a vector or hybrid implementation that we have not built.

“We have two synthetic source documents: the replacement policy and the recall notice. The answer should retain an identifiable citation so we can inspect its basis. A citation is not a truth stamp, and retrieval does not authorize ownership of an order.

“Now let us make the diagram earn its place by running the customer request.”

[At 15:00 advance to slide 9. Show its question and screen route, then use its exact switch cue.]

Slide 09 — Demo 1 — Make the answer useful

Clock 15:00–27:00 · 10 minutes 30 seconds demonstration + 1 minute 30 seconds transitions.

[SWITCH OUT — show this card for 15 seconds. Say the bridge, then Alt+Tab to the prepared terminal at the foundry-showcase repository root. Do not start a deployment.]

“Let us leave the slides and ask the released agent the customer's replacement question. Watch for three things: the business facts, the policy citations and a responsible next step.”

[15:15–15:45: send one fresh request using the command below. It selects only replacement-recall, pins DEMO v1 and refuses cached/replayed output. Set up the environment before presenting. Do not paste credentials.]

```
$env:PYTHONIOENCODING = "utf-8"
$python = ".\agent\src\.venv\Scripts\python.exe"
$source = ".\agent\src"
$out = ".\.runtime\live-" + (Get-Date -Format "yyyyMMdd-HHmmss")
& $python .\scripts\capture_cases.py `
--project-endpoint "https://fdy-volt-citadel-261111.services.ai.azure.com/api/projects/volt-demo" `
--expected-version 1 --dataset "$source\.foundry\datasets\volt-regression.jsonl" `
--case replacement-recall --fresh --out $out --pause-seconds 0
```

“This is a new request, not our saved demonstration output. The customer is outside the standard return window, but that is not the same question as recall-related replacement assessment.”

[After 30 seconds, if the call is still pending, announce the recorded fallback and open Recorded evidence.html → Released DEMO v1 → replacement-recall. Do not submit a duplicate. Keep the pending request distinct from the recording. Only narrate the following result after seeing it.]

“The result is eligible for human assessment, supported by the order, recall and policy evidence. It does not approve a refund or change the order. The structured decision matters as much as the wording.”

[15:45–18:00: inspect the actual answer, decision and citation identifiers. The fresh output is in the new .runtime/live-... directory. If showing a recording instead, keep its recorded banner and version visible. Do not claim that a new response already has an exported App Insights trace.]

“Now we will look behind the answer. A tool listing tells us what was configured. The captured call and result tell us what executed. We need both.”

[18:00–21:00: switch within the demo to the prepared Foundry tab for volt-demo. Inspect volt-support v1 and Toolbox volt-support-tools v1. Show three curated business operations plus the IQ retrieval tool. Do not accidentally show DEV Toolbox v2 and describe it as DEMO's binding. Inspect the response's observed tool names; use the saved native protocol output only with its recorded identity stated.]

“The backend supports additional catalog and compatibility operations, but this support agent gets a smaller capability set. That is deliberate. The Toolbox is not an invitation to expose every API.

“The model is not inventing an order status or a batch recall. The business tools supply those facts. The policy source then explains the assessment path.”

[21:00–24:00: show the prepared Knowledge view for volt-support-knowledge and the two synthetic source documents. Use knowledge/replacement-policy.md and knowledge/recall-notice.md in the editor if portal navigation is slow; those are source documents, not a new retrieval. Compare the returned references VOLT-REPLACEMENT-2026-11 and SYNTHETIC-RECALL-001 with the answer. Never manufacture a passage URL.]

“This is the distinction between an answer and an inspectable answer. The source must actually support the claim. It does not become correct merely because there is a citation icon.

“In a real application, authorization of the customer and their order would be another boundary. Our synthetic fixture does not implement that customer-ownership check.”

[24:00–26:30: switch to the prepared editor. Show only agent/src/main.py, settings.py and response_contract.py, one focused region each. Identify Agent Framework/Responses, the pinned Toolbox connection and the structured response boundary. Use the current folder tree. Do not read the entire implementation.]

“This is our code, running on Foundry's hosted runtime. Copilot helps us develop it; it is not the customer-facing runtime. The model connection goes through our governed route, with no silent direct-model fallback.

“We have now seen the customer outcome and the capabilities behind it. The next question is what stops this useful agent from reaching beyond its intended boundaries.”

[RETURN — at 26:30 Alt+Tab back to PowerPoint, still on slide 9. Advance once to slide 10. Finish the screen switch by 27:00. Do not continue to Memory or Evaluations yet.]

If the call fails or is slow: “The new request has not produced a verified result here. I will show a clearly labeled recorded DEMO response, not claim this is the live outcome.” Preserve the new output/error; never remove --fresh.

If the answer says “I'm escalating”: “That wording is advisory; no ticket or handoff was created. Our business metric does not validate every natural-language action claim.” Keep the recorded text unchanged.

Slide 10 — One gateway. Explicit access contracts.

Clock 27:00–28:00 · 1 minute slides.

“Back to the architecture. We have seen how the agent reaches facts and policy. Useful does not yet mean governed.

“Citadel's Governance Hub puts an explicit access contract around the model and business-tool calls routed through it. We can inspect permitted models and consumption controls instead of relying on the agent to restrain itself.

“That boundary has limits. The gateway contract is not customer order ownership, not a universal dollar cap, and not proof that all service-internal Memory or evaluation traffic follows the same route.

“In Demo 2 we will inspect the actual contract, then separate access control from content filtering and business rules.”

[Debrief Demo 1 before advancing. Stay in the deck through slides 10–12. Switch only at slide 13.]

Slide 11 — Enforce the right boundary

Clock 28:00–29:00 · 1 minute slides.

“These are different boundaries. Authorization asks whether this consumer may use the capability. Content guardrails inspect supported content boundaries. Business eligibility decides what the facts permit.

“A fluent refusal does not prove a platform control fired. We want the native error or enforcement result and its policy context.

“Our harmless demonstration marker produced HTTP 400 with content_filter at the input boundary. We will inspect that verified result and the attached policy, not provoke a dangerous request or promise universal safety.

“The useful request must still work. Governance is not simply blocking everything.”

[Point to the separate boundaries. Advance to slide 12; do not switch windows yet.]

Slide 12 — Remember preferences, not authority

Clock 29:00–30:00 · 1 minute slides.

“Memory is helpful when it remembers a preference, not when it becomes an authority for policy or permissions. Our narrow preference is concise answers with the next step first.

“The verified DEV demonstration used four separate conversations: remember, recall, forget and confirm absence. It retained only a synthetic user-profile preference.

“A new conversation is not a new identity. The right side of the diagram is the next validation task, not a result we have proved: cross-user isolation needs a second genuinely authenticated caller.

“Memory stays off in DEMO. Now let us inspect access, enforcement and continuity together.”

[Advance to slide 13 at 30:00. The demo card is the only switch point for this governance block.]

Slide 13 — Demo 2 — Govern the capabilities

Clock 30:00–45:00 · 13 minutes 30 seconds demonstration + 1 minute 30 seconds transitions.

[SWITCH OUT — show the demo card, say the following bridge, then Alt+Tab to the first prepared surface. Stay out of the deck for the whole block.]

"Let us leave the slides and inspect three distinct controls: permitted access, actual content enforcement and a deliberately narrow preference memory."

[Main route: inspect live configuration and previously completed, identified proofs. There is no verified restricted-consumer denial drill; do not imply one. For Memory, project only the redacted recordings/memory.json summary. The four-session command below is an optional live branch only after timing/redaction rehearsal; do not display its private IDs or scope. Do not rerun it merely because it appears in the notes.]

30:30–34:30 · Azure portal / APIM

[Switch to the prepared APIM product/subscription and policy views. Show only redacted contracts, the approved model route and token limits. The permitted route has execution evidence. A restricted-consumer denial has not been verified; do not run a guessed request or display subscription keys.]

"First, let us inspect an allowed request. The useful path should still succeed. Governance that blocks everything is not a useful product.

"Here is the configured access contract. We have verified the permitted route, but we have not completed a separate restricted-consumer denial drill. I am not going to turn a configuration screenshot into a claim of tested rejection."

[Inspect the permitted-route evidence and configured policy. Explain the intended deny boundary conceptually. There is no denied-request result to show in the current main route.]

"A proper denial test would need the actual gateway status and correlation to this contract. The agent saying 'I cannot do that' would not establish that authorization was enforced.

"The model route and business-tool route also have different contracts. Do not assume one subscription, one policy or one budget magically covers every service-internal call."

[inspect the redacted policy and route identity. If the latest gateway probe is shown, identify `VOLT_GATEWAY_OK` as a gateway-route probe, not proof of the entire customer workflow.]

"A successful model gateway probe establishes that route. It does not establish that the agent used the route in every call, that the tool contract works, or that every internal platform operation shares the quota.

"API Management Developer SKU is also a demonstration choice, not a production availability design. Its baseline cost and the costs of Search, hosting and model usage need separate consideration."

34:30–39:30 · Foundry guardrail / completed enforcement proof

[switch to Guardrails and the recorded proof. **REHEARSAL BINDING RB06 — verified:** the harmless `VOLT_DEMO_BLOCK_MARKER_261111` probe returned HTTP **400**, error code **content_filter**, type `content_safety_error`, at the **input** stage. The local proof is `.\.runtime\volt-guardrail-proof.json` from the demo repository root; request ID `088298eff265d56cd2438f8dbd2dcbb2`; `native_filter_observed` is true. This is platform enforcement, not a model's refusal text.]

[Separate post-promotion DEMO proof is also verified: HTTP 400 / content_filter, `.runtime\volt-demo-guardrail-proof.json`. Keep its project/version identity separate from the earlier DEV probe. The offline viewer states the recorded control result; it is not a new moderation call.]

[Rehearsal-only command, not the main stage action: `.\agent\src\.venv\Scripts\python.exe .\scripts\probe_guardrail.py --project-endpoint https://fdy-volt-citadel-261111.services.ai.azure.com/api/projects/volt-dev --agent volt-support --out .\.runtime\volt-guardrail-proof.json`. Preserve the existing proof rather than overwrite it. The probe targets the named agent without a version argument: record the resolved serving version if separately rerun. Do not assume a new deployment inherits the old proof.]

"Here is the attached policy, and here is the completed harmless-marker probe. It was captured before the session. Its platform enforcement result is different evidence from a model choosing to refuse."

[inspect the prepared proof, explicitly identifying it as captured before the session, or run the rehearsed probe once. Open `http_status=400` and `error.code=content_filter`. Inspect the input-stage message and request ID, rather than displaying only a short refusal sentence.]

"This is the specific evidence we need: the supported boundary acted on the configured fixture. The block is attributable to a control, rather than inferred from the final prose.

"Compare that block with the legitimate request from Demo 1 and the identified prior permitted captures. Safety cannot mean stopping all useful traffic. We want permitted utility as well as a specific, attributable block."

[Compare the actual allowed outcome and recorded block without issuing an extra request. If tool-input or tool-output moderation is not verified, do not demonstrate or claim it.]

"There is a limit to what this test proves. It demonstrates this configured block at this boundary. It does not prove that every harmful request is detected, that every tool is covered, or that the application cannot violate a business rule.

"That is why the business contract remains separate. An allowed, well-grounded request can still end with the wrong structured workflow decision. The guardrail cannot make `recommendation` agree with an eligibility result of `eligible`. Content enforcement is not a replacement for business correctness checks."

[inspect the correlated evidence, then reset the client to the normal synthetic request. Avoid opening a broad safety catalog.]

"The practical takeaway is to name the boundary in your test. 'The agent was safe' is too vague. 'This configured control rejected this fixture while the permitted request still succeeded' is narrower, inspectable and useful.

"We have controlled what the agent may do. Now we need to control what it carries from one conversation to the next."

39:30–44:30 · DEV Memory / verified single-caller lifecycle

[**RB07 single-caller lifecycle PASSED.** Exact command below is read from the showcase README; it explicitly pins DEV v5. Human timing/display rehearsal is still required. Private evidence remains `.runtime\memory-proof-v5-pinned\summary.json` plus four records; never project or copy raw IDs/scope. The offline viewer intentionally omits private Memory records. DEV v6 and DEMO v1 have Memory off.]

[From the demo repository root, documented Memory command: `.\agent\src\.venv\Scripts\python.exe .\scripts\probe_memory.py --project-endpoint https://fdy-volt-citadel-261111.services.ai.azure.com/api/projects/volt-dev --expected-version 5 --out`

[`.\.runtime\memory-rehearsal`. Do not run it during artifact preparation. For a live session, hide raw identifiers and preserve the preview metadata warning.]

"For this chapter, the verified Memory run uses a DEV v5-bound session. The DEV endpoint's v6 and the deployed DEMO v1 both have Memory off. I will distinguish this pinned Memory proof from the business candidate and DEMO release."

"The four-session run completed before this session. I will describe its verified observations, using only redacted material, rather than imply that a new live command has just run. In the first session, the authenticated caller saved the synthetic response-style preference and its exact native content was read back."

[session/conversation 1 — the native remember operation creates the item, and its exact stored content is read back. The actual `remember_response_style` tool executed. Display only the approved synthetic preference text and the verified result; redact native item identity and caller scope. Do not invent the stored string or show raw private JSON.]

"We checked the actual native stored content, not just whether the assistant promised to remember it. The item is deliberately narrow; product facts and policy still come from authoritative systems."

[session/conversation 2 — a separate new session and conversation for the same authenticated caller recalls the saved preference. The retrieval was verified. Show a redacted result, not the scope or native identifiers; answer style alone is not the proof.]

"This is a new conversation with the same authenticated caller. The retrieval evidence demonstrates continuity. Concise wording by itself would not."

[session/conversation 3 — forget the actual native item and confirm native deletion. The operator uses its native identity privately; the presentation must not display or copy the ID. Retain the distinction between an acknowledgment and verified deletion.]

"Now the native item is forgotten and deletion is confirmed. This is stronger evidence than a response that merely says 'I forgot.' TTL is a retention backstop, not a substitute for explicit deletion."

[Deletion scope: forgetting deletes the native Memory item only. It does NOT erase historical conversations or telemetry. Say this explicitly if asked about retention or privacy; do not imply those stores were cleared.]

[session/conversation 4 — a fourth separate session/conversation reports no saved preference. This completes the verified single-caller content/persistence/deletion lifecycle. It is not a test of another user's isolation.]

"A fourth new conversation now reports no saved preference. We have verified content, continuity and deletion for one authenticated caller. We have not proved cross-user isolation; that needs a second genuine identity and a verified scope-to-user binding.

[explain the version warning without revealing session identifiers. SDK/CLI use `extra_body.agent_session_id`; native `get_session` confirmed v5 and the returned session identity matched. The actual `remember_response_style` tool ran. Keep the conflicting preview `agent_reference` warning visible; no raw identifier values belong in this material.]

"The version metadata needs care. Native session lookup confirmed v5, the returned session identity matched, and the Memory tool actually ran. SDK and CLI passed the session binding correctly. The preview response's agent reference still showed the current endpoint v6. That field is misleading here; we preserve both observations and warn rather than call the metadata clean.

"The required permission was Foundry User for the DEV project's managed identity at account scope. OpenAI User alone was not enough. That account-level scope is not a hard DEV/DEMO production boundary.

"This capability is persisted context, not model training or an authorization system. A remembered preference must never override the recall record, the policy or the consumer's permissions. Its model traffic, and evaluation model traffic, are not automatically under our customer APIM token budget.

"That felt more helpful. The next question is how to measure whether the agent actually is better."

[RETURN — at 44:30, Alt+Tab back to the demo card and advance once to slide 14. Read the debrief on that slide before continuing. Finish this demonstration by 45:00.]

If the denial does not occur: "The restricted call was not rejected as expected. That is a failed access-control demonstration. We will not describe it as protected until the actual contract and enforcement are corrected." Use the verified capture only if it exists and is explicitly labelled.

If the policy pane will expose a secret: "We will use the redacted evidence view. Credential material is not needed to understand or prove this control."

If the marker is not blocked: "The policy exists, but this request did not demonstrate enforcement at the agent boundary. We have found a gap between configuration and runtime behavior." Stop the safety claim; do not force the model to refuse with additional prompt instructions.

If the event takes too long: "I will switch to the verified capture of the same policy and version. This is a prepared enforcement result." If none exists, say: "We can explain the boundary design, but we do not yet have runtime proof."

If the bound live command has not been rehearsed on the presentation setup: "The v5-pinned command and prior proof are available, but this live setup has not been rehearsed. I will use the verified redacted summary rather than expose identifiers or accidentally invoke a Memory-disabled version."

If a new live attempt fails: "This new attempt did not reproduce the verified lifecycle. We will preserve its failure and distinguish it from the prepared four-session proof." Do not infer persistence from short wording.

If asked about caller B: "We have proved the lifecycle for one authenticated caller, not cross-user isolation. The second genuine identity and scope/user binding test remain pending." Never replace that test with a manually typed user ID or another conversation.

If deletion is delayed or fails: "The forget request has not yet produced a verified absence. I will not claim deletion until the read-back confirms it." Record the unresolved check and move on at 44:30.

Slide 14 — Evidence before confidence

Clock 45:00–46:00 · 1 minute slides.

“Back from the governance checks: access, filtering and memory boundaries are different controls. None of them guarantees the agent made the right business decision.

“A coherent answer can still be wrong in the field a downstream workflow reads. For Volt, our business contract checks the structured decision, the observed tool evidence and a policy reference when required.

“The original hosted baseline passed eight of nine cases. The failing response sounded correct but said recommendation where the eligibility tool said eligible.

“We will inspect that failed item and the corrected candidate on the same nine cases. A completed evaluation grades the captured responses; opening it does not rerun the agent.”

[Stay in the deck. Advance to slide 15 for the optimization decision, then slide 16 launches Demo 3.]

Slide 15 — Optimize configuration. Review the change.

Clock 46:00–47:00 · 1 minute slides.

“Optimization needs an objective, bounded experimentation and human review. We allowed instruction-only search with separate training and validation prompts, fixed model and tool permissions, and a maximum two-candidate budget.

“The actual job found no accepted improvement. It retained its baseline. That is a valid outcome to discuss, not something to disguise with a green completion status.

“The business-contract defect was repaired separately in code. We must not credit that repair to Optimizer or compare its task-adherence score with a different business metric.

“Now let us read the native results and decide what the evidence really supports.”

[Advance to slide 16 at 47:00. This is inspection of completed runs, not a new optimization or evaluation job.]

Slide 16 — Demo 3 — Measure before improving

Clock 47:00–57:00 · 9 minutes demonstration + 1 minute transitions.

[SWITCH OUT — show the demo card, say the following bridge, then Alt+Tab to the first prepared surface. Stay out of the deck for the whole block.]

"Let us inspect the native baseline, its failed item and the corrected candidate, then see whether Optimizer actually accepted a better configuration."

[Do not start a job. Open the already-completed report links below. The successful Optimizer run took 18m23s; its result is no accepted improvement. Do not click Deploy best candidate.]

47:15–51:30 · Foundry Evaluations

[open the actual native report: https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/evaluations/eval_5a3774acf4f347e1bd35ae30375d0f2d/run/evalrun_28da2cdf28224eb799a9a2e3d9f8672e . Verify Completed status and the business_contract v1 totals. Open failed item 1, `replacement-recall`. The saved baseline replay is `recordings\baseline-v1.json`, commit `78b54e4`; it is captured evidence, not a new invocation.]

"This is the completed run and this is the failed replacement-recall item. We can inspect it in the native report.

"The eligibility tool returned eligible. The model's structured decision says recommendation. The customer-facing text and citations are right, but the workflow contract is not. That is why this item failed even though all four tools executed successfully."

[inspect failed item 1 and its captured tool/decision evidence, then one passing recall case from the same report. Read actual case labels and verdicts; do not substitute a coherence metric. Show that the replay source is the captured hosted baseline. Do not claim that opening the report reran the agent.]

"An aggregate can hide an important failure. Eight passing cases do not cancel this workflow mismatch. We need a clear release consequence for a protected business-contract failure.

"A coherence judge might like the prose. That would be a different measure, not a way to turn this failed business decision into a pass."

[open the completed candidate v6 report: https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/evaluations/eval_5a3774acf4f347e1bd35ae30375d0f2d/run/evalrun_d6967759ba834c51b26a39296474ff0f . Confirm **9 passed, 0 failed, 0 errored**, each of the nine rows passed, same pinned evaluator v1 and unchanged nine-case dataset. This grades the captured real v6 responses. No separate holdout or server-side rerun is implied.]

"Now the actual comparison: v1 passed eight of nine; candidate v6 passes all nine completed-response business cases, using the same evaluator version and unchanged nine-case dataset. We are grading real v6 captures, not re-scoring the old baseline and not asking Foundry to rerun the agent server-side.

"There is an important reliability qualification. One initial model-service invocation failed. That failure was preserved, and the request was retried once. The nine completed responses then passed business grading. Nine of nine is not a claim of one hundred percent operational availability, and zero evaluation errors does not erase the invocation failure.

“The candidate's native code hash matches the packaged ZIP. We will connect that artifact identity to the PR shortly. Passing this business comparison is evidence for review, not permission to merge or promote.

“Memory can contaminate a test if a prior conversation leaks into the next case, so evaluation needs isolated or reset state.

“A reviewed dataset is still a sample. Passing it is evidence, not a proof of every future request. Generated cases can help, but domain review supplies the expected behavior.

“Now we can use this contract to judge optimization instead of treating a higher score as an automatic release.”

51:30–56:30 · Foundry Optimizer / completed job

[open native retry report https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/agents/volt-support/optimization/opt_bcc5008122ae4a88ab10bfa428ecf616 . Portable prepared result: `recordings\optimizer.json`, rendered in `Recorded evidence.html`. The full local `.runtime\volt-optimizer-retry-result.txt` is optional operator evidence, not required by the viewer. Identify this as a completed prior job, not a live search.]

[Verified portal labels: **Optimization outcome: No improvement**, **Succeeded**, **Candidates 0**, baseline/best **0.75**, elapsed **18m23s**, total tokens **207.6K**, baseline-only/stall-limit warnings. The **Deploy best candidate** button is visible: **DO NOT CLICK IT**. It would deploy the retained baseline, not an accepted improved candidate. Use the completed run; never wait eighteen minutes on stage.]

“This job completed before the session. We are inspecting its actual result, not waiting for a search to finish live.

“Read these fields together. Status says succeeded, but termination says candidate screening stalled. The baseline and best task-adherence scores are both zero point seven five. Completed candidate count is zero. Only the baseline came back.

“So the job reached a terminal outcome; it did not demonstrate improved quality. Maximum two was the budget, not evidence that two candidates completed. There is no accepted configuration diff to apply.”

[inspect status, termination, baseline/best score, completed candidate count and warnings. Show the no-accepted-improvement warning. Do not fabricate an accepted candidate, a holdout gain or an applied configuration. Keep the 0.75 task-adherence metric separate from the business-contract 8/9 and 9/9 counts.]

“There was also a real capacity issue before this retry. The first native job failed during candidate evaluation. APIM logs showed upstream model rate-limit responses, followed by Citadel circuit-breaker service-unavailable responses. This was not merely a local watcher or DNS failure.”

[prepared initial-job evidence: `opt_69c51eb68b4a43bc8c9d21b748f190c0`, report https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/agents/volt-support/optimization/opt_69c51eb68b4a43bc8c9d21b748f190c0 . Root evidence is actual upstream 429s, then circuit-breaker 503s. Do not rerun or change capacity. The mini deployment previously read 10 RPM / 10,000 TPM, below the 20,000-TPM application contract. User-approved capacity 10→50 read back 50 RPM / 50,000 TPM, same DataZoneStandard, model version and RAI policy. APIM 20k TPM / 500k daily token limits stayed unchanged.]

“The backend had ten requests per minute and ten thousand tokens per minute available, even though the application contract allowed twenty thousand tokens per minute. A gateway allowance does not create backend capacity.

"The user approved increasing capacity to fifty requests per minute and fifty thousand tokens per minute, without changing the model, deployment SKU or safety policy. The application token controls stayed unchanged. Then the user authorized one retry—not an open-ended search.

"That was a retry of the native Optimizer job, separate from the single invocation retry disclosed in the v6 business captures."

"Even without an accepted improvement, this job consumed resources. Its reported usage and latency belong to this run, not to an SLA or a cost-saving promise."

[inspect reported counters: agent tokens **143,266**, evaluation **49,373**, optimization **14,979**; agent average **20.682 seconds across 20 calls**. These are reported run metrics, not a pricing calculation, all-traffic APIM-budget claim, customer SLA or guarantee. Training/validation were separate, but there is no accepted candidate validation improvement to show. No candidate was applied.]

[Aggregation warning: the portal baseline table displays average **11.1s / 4.6K tokens**. That is a different aggregation from the job-wide agent average 20.682s across 20 calls. Do not mix the figures or use them as a before/after improvement. Prefer the no-improvement outcome over a latency detour.]

"The decision is straightforward: no accepted improvement, so retain the Optimizer baseline. We do not confuse its task-adherence score with business correctness, and we do not credit it for the separate code repair.

"Now return to the exact failure we can explain: a structured decision that disagreed with the observed eligibility result. That is where reviewed regression evidence and a narrowly scoped PR belong."

[RETURN — at 56:30, Alt+Tab back to the demo card and advance once to slide 17. Read the debrief on that slide before continuing. Finish this demonstration by 57:00.]

If a new run is queued: "The new run has not completed. I will inspect our completed hosted baseline and identify it. I will not replace the new result with local CI numbers or imply that the baseline is the corrected candidate."

If the native report cannot load: "The report is unavailable in this view. I will open the saved real baseline capture and keep its identity visible. The completed native result is eight passed, one failed and zero errored, but this local replay is not a new evaluation or agent execution." Use `baseline-v1.json`; do not manufacture a portal screenshot.

If business and quality verdicts disagree: "That disagreement is useful. A fluent response can fail the business contract. Our release rule must respect the business constraint rather than average it away."

If asked why succeeded produced no improvement: "Succeeded is the job status. The termination reason, zero completed candidates and baseline-only result establish that no optimized candidate was accepted. We applied nothing."

If asked to try again: "The user authorized one retry and it has completed. We do not launch another job or change capacity during this presentation."

If the portal is slow: "I will open the saved result for this exact retry. It is prepared evidence and still shows no accepted improvement." Use `Recorded evidence.html` or `recordings\optimizer.json`; do not reinterpret a watcher problem as the native job outcome.

Slide 17 — Operation teaches DEV what to test

Clock 57:00–58:00 · 1 minute slides.

“Back from the reports: fluent output was not enough, and a completed optimization job was not an improvement. Now we have a precise repair question.

“The original incident is real: eligible tool output became a recommendation decision. Its evidence is a native captured response and failed evaluation. We did not find its reported identifier as an exported App Insights span, so we will not pretend that we did.

“A trace or capture is evidence, not a label. A person reviews the expected behavior before it becomes a regression.

“The fix must preserve missing and unknown information, not hard-code every request to eligible.”

[Keep the feedback-loop diagram visible, then advance to slide 18. Demo 4 starts on slide 20, after the release and evidence-chain diagrams.]

Slide 18 — A PR carries code and behavioral evidence

Clock 58:00–59:00 · 1 minute slides.

“A PR should carry both code and behavioral evidence. We need the reproduction, the reviewed expectation, the candidate identity and the checks that judge it.

“The actual repair was made with local Copilot assistance. Its PR is already merged. We will inspect that real history, not stage a second merge or invent a cloud coding-agent run.

“The operator-run promotion gate checked the exact candidate and uploaded the same saved ZIP. That is not a claim that GitHub branch protection or protected-environment approvals are configured.

“Deployment completion is not the finish line. We must verify the serving version and its independent behavior.”

[Point from DEV through candidate/gate to DEMO. Advance to slide 19 without leaving PowerPoint.]

Slide 19 — Follow one fix from trace to customer

Clock 59:00–60:00 · 1 minute slides.

“This is the chain we will follow in the final demonstration: observed mismatch, reviewed case, issue and PR, candidate artifact, gate and serving version.

“Each link must identify the same change. A pile of green results from unrelated versions is not release evidence.

“DEMO received the tested ZIP without a rebuild, then produced its own nine-case evaluation. Its Memory setting remains off and its access contracts are separate from DEV.

“We will finish with another genuinely new customer request. If it differs or fails, we preserve that result rather than borrowing a passing recording and calling it live.”

[Advance to slide 20 at 60:00. Point to its three stops and return target before switching to the first evidence view.]

Slide 20 — Demo 4 — Close the loop

Clock 60:00–78:00 · 16 minutes demonstration + 2 minutes transitions.

[SWITCH OUT — show the demo card, say the following bridge, then Alt+Tab to the first prepared surface. Stay out of the deck for the whole block.]

"Let us follow the captured failure into a reviewed test and actual Copilot-assisted patch, inspect the release evidence, then send one new request to the serving agent."

[Inspect the existing closed issue and merged PR; do not create or merge them again. The PR contains historical source paths, while the cleaned checkout uses agent/src and evaluations. The deployed ZIP and native evaluation identities remain unchanged. For the final command, use a new timestamped output directory again. If it has not returned after 30 seconds, announce the recorded fallback; no duplicate request. Use the last minute to summarize the evidence chain and switch back by 78:00.]

60:30–64:00 · Captured incident and reviewed regression

[Switch to baseline-v1 in `Recorded evidence.html`, or the native failed case/report. **RB10:** inspect the same replacement-recall response, tool sequence and decision. There is no verified exported App Insights span for this incident. The lifecycle diagram's trace flywheel is the conceptual pattern; the actual evidence here is native capture plus evaluation.]

"This is the request and this is the version that handled it. We will start with the recorded tool result and structured decision, not an AI-written incident summary.

"For this incident we have the native captured response, its tool sequence and a failed native evaluation. We did not find its reported correlation ID in exported App Insights. There is also no automatic Insights finding. Neither gap should be hidden by calling a manual issue an Insight."

[inspect the captured native request and order/recall/eligibility/IQ calls, actual eligible result and recommendation decision. Keep response/version identity visible; do not claim an App Insights operation for this historical capture.]

"The customer did provide the order and batch. The eligibility tool returned eligible, meaning eligible for human assessment in this synthetic flow, not an instruction to issue a refund.

"The model emitted recommendation in the structured decision. A downstream workflow reading that field does not get the same outcome as the correct natural-language explanation. That is the contract mismatch we need to repair."

[replay the captured real baseline and correlate it with failed item 1 in the native report. State that this is a saved execution, not a new server-side agent run. An optional live repeat uses the exact slide-7 request, but cannot replace the original evidence if the model produces a different decision.]

"This replay preserves the failure even if another live answer varies. It is not a fabricated result. It is the actual hosted output the native evaluator scored.

"We know what disagreed. We should not invent the root cause from the transcript alone. The code review must find how the final decision is derived and how the authoritative tool result should constrain it."

[**RB11** is the replacement-recall row in `agent\src\.foundry\datasets\volt-regression.jsonl`, plus `evaluations\business_contract.py` pinned as evaluator v1. Inspect expected eligible against captured recommendation, then the merged PR. DEMO's completed result is a separate fresh capture set, not a reuse of this baseline.]

“A trace is evidence, not a label. The native case identifies the mismatch; a person still reviews the regression and its business expectation. For this request, the structured decision must preserve the eligible assessment outcome supported by the tool.

“We must not solve it by hard-coding every response to eligible. Preserve the other eight baseline cases, including missing and unknown information, and preserve the no-mutation boundary.

“The regression needs the input, the expected decision and the provenance of the observed tool results. That is more useful than a screenshot of prose that already looks correct.

“This is the separate, precise repair path. Optimizer accepted no change, so its baseline stayed in place. The business-contract mismatch was addressed by local Copilot-assisted code work and a reviewed PR. That PR has merged and the same artifact reached DEMO. We will inspect its completed independent post-promotion result, not reuse DEV's score.”

64:00–74:30 · GitHub PR, focused code diff and exact-artifact gate

[open issue #1 and merged PR #2. **RB12:** show reviewed head, merge 04a2197, target feature/volt-citadel and successful existing CI/native statuses. Explicitly say main was not changed. Inspect existing records only; do not replay merge/deployment actions.]

[Issue #1 is now CLOSED as completed, with a comment linking the DEMO report. Merged-branch CI run **43065816204** succeeded: https://github.com/SridharArrabelly/foundry-showcase/actions/runs/43065816204 . Keep this separate from earlier foundation CI and native business grading.]

“This issue is our record of the real replacement-recall mismatch. We can connect it to the captured response, the failed native item and the candidate PR.

[inspect the prepared local-assisted diff at PR head `1d01eaae455ac0b2aa39da0f5a57d5543017a5e2`. State that the work was completed before this inspection. Show a local-session record only if it exists; do not invent a transcript, exact original prompt or cloud job. Limit editor time to the decision derivation and relevant regression.]

“Let us inspect the patch, not just the assistant's explanation. Where is the final structured decision derived? How does it use the observed eligibility result? Does it preserve missing, unknown, recalled and non-recalled cases rather than hard-coding a favorable answer?

“We inspect how the regression relates to the original failure and how the normal cases are preserved. The verified native comparison uses the unchanged nine-case dataset; a green check on unrelated behavior would not prove this repair.”

[inspect `.runtime\volt-candidate-package.json` and saved `.runtime\volt-candidate-1d01eaa.zip` from the demo root. The native v6 code hash matches ZIP SHA-256 `ac015b257905e6dbd92f04bc49d6f86c8eb757f22557433c7c2ad47a83508996`. Keep candidate version, PR head and package association distinct from the promotion record. Inspect, do not rebuild or deploy.]

“This is the PR head, and this is the candidate artifact identity. The native v6 code hash matches the packaged ZIP. That is stronger evidence than a label saying ‘latest.’

“Now we inspect the native business result for that candidate. Other PR checks and human approval still answer separate questions. We do not call this the whole release gate merely because the business evaluation passed.”

[open candidate report https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/evaluations/eval_5a3774acf4f347e1bd35ae30375d0f2d/run/evalrun_d6967759ba834c51b26a39296474ff0f . Confirm Completed, v6 captures, **9 passed / 0 failed / 0 errored**, each row passed, same pinned

evaluator v1 and unchanged nine-case dataset. Retain the v1 8/9 report as the baseline, not a substituted metric.]

"The candidate passes all nine completed-response business cases, compared with eight in the v1 baseline. These are real captured v6 responses. Native evaluation grades them; it does not rerun the agent server-side.

"We also preserved an initial model-service failed invocation and retried it once. That operational failure is not erased by nine passing completed responses. Nine of nine does not mean one hundred percent availability, and zero grading errors is not a statement that every invocation succeeded.

"The evidence should identify the source commit, the built artifact or image where applicable, the agent version, the pinned Toolbox, the knowledge revision or snapshot, and the dataset and evaluator versions. Some systems expose these differently; the release record must still make the association explicit."

[inspect merged PR and the operator-run `scripts\promote_demo.py` gate plus recorded deployment evidence. **RB14 is bound to approved promotion:** merged exact PR, GitHub CI, native case counts and expectations, grader hash and source hash are checked before uploading the same saved ZIP. No rebuild. This is operator-run enforcement, NOT a claim of GitHub branch-protection or environment enforcement. Do not execute the script again for the presentation.]

[Documented release invocation from verified static arguments and existing inputs, from demo root: `.\agent\src\.venv\Scripts\python.exe .\scripts\promote_demo.py --package .\.runtime\volt-candidate-1d01eaa.zip --deployment-receipt .\.runtime\volt-candidate-package.json --evaluation-receipt .\agent\src\.foundry\results\candidate-v6\native-evaluation.json --expected-commit 1d01eaae455ac0b2aa39da0f5a57d5543017a5e2 --pull-request 2 --out .\.runtime\volt-demo-release.json --execute`. Record only: do not rerun an already completed promotion. The existing receipt deliberately prevents duplicate execution.]

"The user approved merge and promotion. The PR merged into the feature branch shown here; main was untouched. The promotion script checks the exact merged PR, CI, native case counts and expectations, and the grader and source hashes.

"Be precise about where that gate lives. This demonstration uses an operator-run promotion script. We are not claiming GitHub branch protection or protected-environment enforcement that has not been demonstrated.

"The gate uploads the same saved ZIP rather than rebuilding after evaluation. The release used the documented SDK and CLI multipart code route, because the azd from-package route did not supply the required code-zip metadata."

[inspect DEMO v1/native hash against saved ZIP SHA-256 `ac015b257905e6dbd92f04bc49d6f86c8eb757f22557433c7c2ad47a83508996`, then its distinct completed report: https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-demo/build/evaluations/eval_a0c23c81fa62456fabd5f05a11453cec/run/evalrun_c61cce9dc8774d528841ca069d992255 . Result 9 passed / 0 failed / 0 errored. The actual `.runtime\volt-demo-release.json` receipt now says **runtime-verified**. `verify_demo_release.py` finalized it after checking the live completed native DEMO evaluation, target code hash/configuration and pinned grader. It does not rerun the agent. Operator command is in Appendix C; inspect the completed record, do not execute it on stage. DEMO Memory stays off and runtime role is project-only.]

"The native DEMO code hash matches the saved artifact. DEMO has its own access contracts and connections. The connection names must be distinct across projects because of account-level ownership collisions, while the labels the Toolbox presents remain stable.

“Memory is off in DEMO, and that runtime has only project-scoped Foundry User, not the account-level Memory roles used by our separate DEV v5 demonstration. We do not imply that promoting the business agent promoted all preview demonstrations with it.

“The full nine-case post-promotion run completed with nine passed and zero failed or errored in DEMO's own native evaluation. This is not DEV's result reused. The operator verifier checked that evidence, target configuration and pinned grader before finalizing the receipt as runtime-verified. That is still not GitHub-enforced protection.”

[Presenter-only connection map: `volt-demo-models-LLM`, `volt-demo-business-tools`, `volt-demo-policy-knowledge`; DEMO Toolbox v1. Responses endpoint: `https://fdy-volt-citadel-261111.services.ai.azure.com/api/projects/volt-demo/agents/volt-support/endpoint/protocols/openai/responses?api-version=v1`. Native DEMO guardrail separately returned HTTP 400 / content_filter (`.runtime\volt-demo-guardrail-proof.json`). No automatic rollback drill is claimed.]

74:30–77:00 · Fresh released-agent request and separate recorded trace

[RB15 COMPLETE FOR THE SHOWN CASES — DEMO volt-support v1, same ZIP/native hash, nine fresh runtime cases passed; DEMO native evaluator v1 completed 9 passed / 0 failed / 0 errored. Export: `recordings\demo-v1.json`. Separate fresh golden response `caresp_03f2cedafe9a81ff00ml4CEH7VZPqFrxR1SFFHpglDzRb3u45a` correlates to verified App Insights root `7f4619cba4075a99c6d1b077dad68f69`. Do not conflate it with the unverified old incident span. Memory remains off in DEMO.]

[Switch to the prepared terminal at the **repository root** (`foundry-showcase`). This single-case route was previously exercised successfully. Run it once for a new request. Keep UTF-8 output, a new timestamped directory and --fresh; never reuse an old output directory and call it live. The source cleanup did not invoke or redeploy Azure resources.]

```
$env:PYTHONIOENCODING = "utf-8"
$python = ".\agent\src\.venv\Scripts\python.exe"
$source = ".\agent\src"
$out = ".\.runtime\live-" + (Get-Date -Format "yyyyMMdd-HHmmss")
& $python .\scripts\capture_cases.py `
--project-endpoint "https://fdy-volt-citadel-261111.services.ai.azure.com/api/projects/volt-demo" `
--expected-version 1 --dataset "$source\.foundry\datasets\volt-regression.jsonl" `
--case replacement-recall --fresh --out $out --pause-seconds 0
```

“This command sends a new request to released DEMO version one. It selects only our replacement-recall case, creates a timestamped output directory and refuses cached or replayed output.”

[While the one invocation runs, use the preceding sentence. Do not submit a duplicate. After **30 seconds**, if it is still pending, switch to the explicitly recorded fallback below. This is a pacing threshold, not a latency promise. Only read the next sentence after the actual output shows eligible and genuine policy citations; otherwise use the failure branch.]

“Here is the actual eligible assessment decision and the policy citations. It does not approve or change the order. This one-case check is separate from the completed nine-case release gate.”

[inspect actual returned decision/citations. If time permits, point to the separate, previously verified golden trace in the offline viewer—not a claim that this newly submitted response already has verified correlation. That saved proof is `.runtime\volt-demo-golden-trace.json`, response `caresp_03f2cedafe9a81ff00ml4CEH7VZPqFrxR1SFFHpglDzRb3u45a`, root `7f4619cba4075a99c6d1b077dad68f69`, UTC `2026-11-11T17:19:09.3446407Z`, duration 23.4094s: one observation, not an SLA.]

[If showing the older recorded golden answer, its “Next step: I’m escalating this to human support” text must stay unchanged. Read the advisory-wording caveat below. The newer fresh rehearsal answered more

cautiously, but it does not retroactively fix this recorded natural-language limitation. Neither response proves a ticket/handoff; no such integration exists.]

"Our saved golden request has separate verified App Insights correlation. That trace proves observability for that request—not automatically this new response, and not the original incident."

[The fresh golden response now has an export of **43 verified correlated spans** in `recordings\demo-trace.json`. The offline viewer includes only whitelisted span names, durations and recorded success, not raw telemetry properties. This remains one fresh golden request, not the old incident or a monitor configured over all traffic.]

"Follow the chain. Native captured response and tool evidence exposed the old mismatch. The reviewed repair passed DEV checks, the user approved merge and promotion, and the operator gate uploaded the same ZIP. Fresh DEMO cases passed their own evaluation, and a separate golden request proves current observability.

"Every link should refer to the same change. A pile of green screenshots from unrelated versions is not this chain.

"This is how operation teaches development—and how we distinguish a promising candidate from a verified synthetic DEMO release."

[RETURN — at 77:00, Alt+Tab back to the demo card and advance once to slide 21. Read the debrief on that slide before continuing. Finish this demonstration by 78:00.]

If asked for the baseline App Insights span or automatic finding: "The old correlation ID was not found in exported App Insights and no automatic finding is verified. The native response/tool sequence and failed evaluation are our incident evidence. Fresh DEMO observability is a separate proof."

If a live repeat does not reproduce: "This new answer differs from the captured failure. That is not proof of a repair. We will use the identified v1 replay and the completed native report to inspect the actual incident." Never edit a new answer to recreate the failure.

If the native portal is slow: "I will use the offline recorded viewer. It shows the captured model answer and tool names, not a simulated portal or an invented span."

If asked to show the cloud job: "There is no executed cloud coding-agent job to show. Dispatch failed before start. The fix used local Copilot assistance in an isolated worktree, and this is its actual PR diff."

If Copilot produces an incorrect patch: "The proposed change does not meet the reviewed contract. We reject it. A development assistant is not an approval authority." Use the already reviewed fallback only if one exists.

If a gate fails: "This candidate is not releasable. The gate has stopped it. We can inspect the failure or use the previously approved candidate, but we do not override the evidence to finish a demo."

If the release succeeds but the serving version does not match: "Deployment completion and serving identity disagree. We have not proved promotion. We will not present the customer result as this candidate until that identity is resolved."

If asked whether the fix is released: "It is deployed and verified in synthetic DEMO as version one, using the same ZIP. Nine fresh runtime cases and DEMO's own native business run passed. That is not a claim about customer production, universal correctness or availability."

If the fresh command is slow or unavailable: "The new invocation has not completed here. I will use our preserved DEMO capture, clearly marked recorded evidence—not call it this live response." Open `Recorded evidence.html`, choose Released DEMO v1 and replacement-recall; keep the banner visible. Do not rerun the command or change `--fresh` to reuse output.

If the recorded answer says "I'm escalating": "That wording is advisory; no ticket or handoff was created. Our nine-case metric validates decision/tool/reference, not every natural-language action claim." The backend is read-only. Do not edit the recorded output or equate the newer cautious wording with erasure of the earlier limitation.

If fresh output fails or differs: "This new request has not reproduced the rehearsed result. We will preserve its actual output separately from the completed nine-case evidence." Do not overwrite the new directory, fabricate eligible, borrow another response's citations, or turn the prior trace into this request's trace.

If the displayed evidence has a different identity: "This is not the verified DEMO run or artifact. We must resolve that identity before claiming a post-promotion result." Do not infer success from a similarly named agent.

Slide 21 — Useful. Governed. Measurable. Improving.

Clock 78:00–80:00 · 2 minutes slides. Then 80:00–90:00 Q&A.

“Back from the final demonstration, we can connect capability, governance, measurement and reviewed change. That is the full value of the two loops.

“Four ideas to take back to your next agent.

“First, useful. Give the agent authoritative capabilities, not just a larger prompt. Our tools established business facts, and IQ supplied policy evidence we could inspect.

“Second, governed. Name the access contract and the boundary. Authorization, content controls and business rules solve different problems. Memory should preserve appropriate continuity, not become a hidden policy or permission store.

“Third, measurable. Correctness is a contract. Fluency and coherence are valuable, but they do not prove recall correctness. Evaluate identifiable candidates against reviewed cases, and inspect failures at case level.

“Fourth, improving. Operational evidence returns to development through human review. Copilot can help implement a narrow repair. The PR, evaluation gate and serving-version check keep that repair connected to the customer outcome.

“The point of the two loops is not to slow developers down. It is to preserve what we learn, make changes reviewable and avoid starting from a convincing demo every time.

“We did not just build an agent. We built a way to know what it did, control what it may do and improve it without losing the evidence.

“We have ten minutes for questions. Which boundary in your own application would you want to prove first: knowledge, tools, memory, evaluation or release?”

[Keep slide 21 visible for Q&A. Use hidden slides 18–19 only when the question benefits from their diagrams. Do not automatically advance into hidden backups. If any planned proof remained incomplete, replace the closing “we built” sentence with: “This is the operating model we are building toward; the incomplete proofs we named remain release blockers.”]

Slide 22 — Hidden backup: memory scope

Q&A only · not included in the 80-minute route or the audience PDF.

[Open this hidden slide only for a memory/isolation question. Return to slide 21 afterward.]

“A Memory Store is a service container for persisted context. Scope partitions access to the items we retrieve, but the application must derive and use that scope correctly.

“The store is configured for one narrow synthetic preference, seven-day TTL and user profile only. We verified content, persistence and deletion for one authenticated caller across four separate sessions and conversations. Scope-to-user binding and cross-user isolation still require a second genuine authenticated identity.

“The native v5 session pin was correct. SDK and CLI use extra_body agent_session_id, native get_session confirmed v5, the returned session identity matched, and remember_response_style ran. The preview response's agent_reference nevertheless reported current endpoint v6, which has Memory off. We preserve the discrepancy rather than claim clean version metadata.

“The documented remedy was Foundry User for the DEV project's managed identity at account scope; OpenAI User alone was insufficient. That broader scope and the shared DEV/DEMO account are still not hard production isolation.

“Two conversations for caller A can demonstrate continuity. They cannot demonstrate caller isolation. That requires another authenticated caller, B, and an observed absence of A's preference.

“The same distinction applies to forgetting. A successful delete response is useful, but a read-back establishes that the item is no longer returned. Retention and deletion should be tested separately.

“Forgetting here deletes the native Memory item, not the historical conversations or telemetry. Those have their own retention and deletion responsibilities.”

“This is not model training and not a system of record for recall, policy, identity or authorization. The memory preview also has network constraints; this session is not an entirely private-network design.”

[Current answer: “The single-caller lifecycle passed. Cross-user isolation still needs a second genuine authenticated identity. The bound README command pins DEV v5; DEV v6 and DEMO v1 have Memory off. Preview agent_reference metadata remains misleading.” Never display native Memory IDs or caller scope.]

Slide 23 — Hidden backup: account and project responsibilities

Q&A only · not included in the 80-minute route or the audience PDF.

[Open this hidden slide only for environment, identity or ownership questions. Return to slide 21 afterward.]

“The account and project scopes separate different responsibilities. The account is where model deployments and shared infrastructure are managed. Projects hold the agent-facing resources and runtime context used by teams.

“Our selected account is `fdy-volt-citadel-261111`, with `volt-dev` and `volt-demo` projects in the approved tenant. The resource group is `rg-volt-citadel` in Sweden Central.

“That structure helps us organize responsibilities, but this actual profile is not hard production isolation: DEV and DEMO share the account and the effective managed-identity role scope for Memory is broader than project scope. The intended production boundary needs its own identity, role and release design.

“Memory and evaluation model traffic are not automatically under the customer's APIM token budget. Project organization and a customer gateway policy do not imply a common budget boundary for every service-internal call.

“The public Foundry endpoint is Entra-authenticated in the approved design. Backend storage and Key Vault remain private. A data-zone model deployment is not a promise of Sweden-only processing.

“In a real customer system we would additionally need customer identity and order-ownership authorization, appropriate logging and retention, operational support and a production availability design. We should not confuse a synthetic enablement environment with a complete production compliance posture.”

Appendix A — Complete Q&A answer bank

Why hosted rather than prompt?

"We chose hosted for custom deterministic runtime logic and this integration path. Prompt agents are a good managed option when their orchestration meets the need. Tools, CI and source control do not independently require hosted. The trade-off is owning application code and dependencies in addition to the usage and hosting costs."

Is Copilot the agent runtime?

"No. GitHub Copilot is the development assistant in this route. The customer runtime is Python Microsoft Agent Framework using Responses. The earlier Copilot SDK Invocations implementation is historical, not part of this workshop or proof of hosted optimizer compatibility."

Did the cloud coding agent make this fix?

"No. Cloud dispatch failed before starting because the gh credential was not an OAuth token. The actual fix used local Copilot assistance in an isolated worktree. Issue 1 and merged PR 2 are the records; there is no executed cloud coding-agent job to show."

Does 9/9 mean every invocation succeeded?

"No. Candidate v6's nine completed real responses passed business grading. One initial model-service failed invocation was preserved and retried once. The result is not one hundred percent operational availability, and zero evaluation errors does not erase the invocation failure. Native evaluation grades captures rather than rerunning the agent server-side."

Does Citadel make the solution compliant?

"No. Citadel supplies an implementable governance architecture. Compliance depends on the actual controls, data, identities, network posture, retention, operating procedures and organizational requirements. A gateway deployment does not automatically turn on Defender, Agent 365, Purview or every layer in the blueprint."

Is every hop keyless?

"No. The backend uses managed identity for private storage. APIM reaches the Function with a function-scoped key held in Key Vault. Toolbox uses a managed per-use-case subscription credential to APIM. We need to describe and rotate those credentials according to their real contracts rather than call the entire chain keyless."

Can a user read someone else's order?

"This synthetic demonstration does not implement real customer order-ownership authorization. An authenticated platform caller and an APIM consumer contract are not enough to establish that a real customer owns a particular order. That is a production requirement we would implement and test before using real customer data."

Does the gateway impose a dollar budget on all AI usage?

"No. The configured twenty-thousand-token-per-minute and five-hundred-thousand-daily-token limits are token controls on the routed path, not a monetary cap across the platform. Memory and evaluation model traffic are not automatically under the customer's APIM token budget. Other service processing and idle infrastructure can also incur costs."

What does it cost, and how fast is it?

"We have not published a validated cost-per-case or latency benchmark for this deliverable. The relevant cost includes APIM Developer, Search, hosted compute, Functions, storage, telemetry and model usage, including evaluation and optimization. Latency can include retrieval, multiple tool calls, model work and cold or queued execution. We would measure the representative path with versioned workload assumptions rather than promise a generic saving."

Is all processing in Sweden?

"The selected resource group is in Sweden Central. The model uses a data-zone deployment SKU, so that location does not establish Sweden-only processing. We need to inspect the specific model, service and telemetry data-handling commitments separately."

Does IQ use vector search in this demo?

"No. The implemented path is Azure AI Search with text and semantic extractive retrieval. We show the source-to-knowledge-to-agent architecture without claiming vector or hybrid retrieval or additional federated sources."

Can we replace the human with an automatic refund?

"That would be a different risk and authorization design. Today's tools are read-only and the next step is human assessment. Moving money or changing an order needs explicit transaction authority, customer ownership checks, idempotency, appropriate approvals and its own evaluation and audit evidence."

Can guardrails prevent every unsafe action?

"No. A successful blocklist fixture proves that configured boundary for that input. It does not establish universal jailbreak resistance or coverage of every tool. Content controls complement authorization, deterministic business rules, least privilege and human approval."

Does memory train the model, and can it store policy?

"Here memory is persisted context retrieved for future requests, not training. We deliberately exclude policy, product facts, customer identity and permissions. Those remain authoritative elsewhere. We test retention, deletion and identity separation rather than trusting a convenient chat experience."

Why not release the optimizer's highest-scoring candidate?

"Because its objective is not the whole release contract—and in this actual run there was no accepted optimized candidate. Baseline and best task-adherence were both 0.75, completed candidates were zero, and nothing was applied. We retained that baseline. The separate business-contract code fix went through the reviewed PR and operator promotion gate."

Did succeeded mean the Optimizer improved quality?

"No. Its status was succeeded, but termination was candidate_screening_stalled. Only the baseline returned. Maximum two was a candidate budget, not two completed candidates. We disclose the no-improvement warning rather than use job status as a quality claim."

Was promotion enforced by GitHub branch protection?

"Not in the demonstrated mechanism. The operator-run promote_demo.py checked the merged exact PR, GitHub CI, native case counts and expectations, grader hash and source hash, then

uploaded the same saved ZIP. That is a real gate, but it is not evidence of GitHub branch-protection or environment enforcement."

How do we know the evaluator is right?

"We review the expected business outcomes and test the evaluator itself against known pass and fail cases. Tool evidence and structured decisions make the contract more inspectable than fluency alone. Evaluator versioning, dataset review and case-level disagreement remain important; no aggregate removes the need for domain judgment."

Why not let production traces automatically become labels?

"A trace tells us what happened, not what should have happened. It can contain an incorrect answer, irrelevant data or sensitive content. We select safe evidence, review the expected behavior and only then promote it into a regression case."

Is the defect real?

"Yes. The captured DEV v1 response had correct prose/citations but the wrong structured decision. The separate local-assisted repair passed DEV v6 cases, merged into feature/volt-citadel, and reached DEMO unchanged. Nine fresh DEMO cases passed their own native evaluation. Main was untouched; no Optimizer candidate was applied."

What happens if the pipeline fails during the session?

"We keep the failure visible and do not release that candidate. If there is a previously approved run of the same repair, we label it as prepared and inspect its identity. Otherwise the release proof is incomplete. Finishing the story is not a reason to bypass a gate."

Appendix B — Shortened delivery paths

If five minutes behind at 45:00: preserve native business evaluation and the trace-to-release chain. Recover three minutes in Demo 3 by showing the Optimizer's exact no-improvement outcome without the capacity/log/token detour. Recover two minutes in Demo 4 by showing the selected failed case directly instead of navigating the full captured protocol. Do not invent a candidate diff or holdout improvement.

If ten minutes behind: use the offline viewer for baseline/candidate/DEMO comparison and keep its recorded-only banner visible. Compress Memory to redacted single-caller proof/limits and the Optimizer to its no-improvement outcome. Preserve the merged PR, exact-ZIP promotion, independent DEMO 9/9 and separate fresh-root observability proof. Keep Q&A at minute 80.

If a whole integration is unavailable: do not spend the recovered time narrating a fictional live proof. Say: "This capability remains a design walkthrough in this environment; the runtime proof is not available." Move to verified evidence in the next chapter. The audience should leave knowing what was demonstrated and what was not.

If only sixty minutes are available for content: use slides 1–6 in ten minutes; customer tools and IQ in twelve; gateway and guardrail in eight; memory in four using prepared evidence; evaluation and optimizer in eight; trace, reviewed fix and release in fourteen; customer proof and four takeaways in four. Keep a separate ten-minute Q&A. Do not provision, wait on optimizer jobs or expand the code tour.

Appendix C — Final rehearsal checklist

- Rehearse the bound RB01–RB15 navigation, commands and fallbacks. Technical identities are filled; the optional RB05 negative-access drill needs real denial evidence or its concept-only branch. No unresolved human choice is silently treated as ready.
- Verify the served runtime is Agent Framework + Responses; do not substitute the historical Invocations implementation.
- Use the commands in slides 9 and 20 for new DEMO requests: UTF-8 output, new timestamped directory, --case replacement-recall, --fresh and --expected-version 1. This invocation route was previously exercised; the revised full-session timing/navigation still needs rehearsal. One case is not the nine-case release gate.
- Retain the verified Toolbox v2 preflight evidence described below; separately verify the hosted agent's correlated invocation and model route.
- Verify the IQ references and passages. Do not infer grounding from service existence.
- Inspect permitted APIM evidence and redacted configuration. A denied-consumer outcome is unverified and is not part of the current stage route.
- Inspect the completed guardrail block and the separate allowed-request evidence, identifying each as live or recorded.
- Rehearse the bound README Memory command with --expected-version 5. Preserve redaction/metadata warnings. Second-authenticated-user isolation remains unproved; offline viewer omits private Memory evidence.
- Open both native reports: baseline v1 8/9 and candidate v6 9/9, same pinned evaluator v1 and unchanged nine-case dataset. Disclose the preserved initial model-service failure and one retry. These grade completed real captures, not agent reruns or availability.
- Open the actual Optimizer retry report/result. Keep succeeded, candidate_screening_stalled, baseline/best 0.75, completed count 0, maximum budget 2 and nothing applied together. No new job or capacity change is authorized by the presentation.
- Preserve the v1 native response/tool sequence, failed native case and baseline-v1.json. Do not claim its reported correlation ID is an exported App Insights span. Show only the separate fresh DEMO root as verified current observability.
- Open merged PR #2: head `1d01eaae455ac0b2aa39da0f5a57d5543017a5e2`, merge `04a2197`, target `feature/volt-citadel`, main untouched. Inspect promote_demo.py checks, same-ZIP upload and DEMO v1 native hash, then the finalized runtime-verified receipt. verify_demo_release.py reads completed native DEMO evidence, target hash/configuration and pinned grader before writing that status. Neither script is GitHub-enforced protection.
- Ensure fallback captures contain real timestamps and versions and are labelled prepared. No synthetic portal screenshot is evidence.
- Check cost and quota headroom, current preview access and permissions. Do not expose secrets or broaden permissions on stage.
- Human 90-minute timing rehearsal is not done. Rehearse transitions, presentation-network sign-in and local Recorded evidence.html before delivery, without rerunning merge or promotion.

- Check the revised demo cards on slides 9, 13, 16 and 20 and return slides 10, 14, 17 and 21 on the actual projector. The cleanup adds visible demo cues; it is not merely a notes-only revision.

Verified Toolbox preflight — presenter preparation only, not a live feature catalogue. The implementation owner has verified native APIM business MCP end-to-end through Foundry Toolbox v2. The curated Toolbox is exactly three business operations plus one IQ tool. Six direct-tool cases passed: recalled CH-0722; missing batch → `needs_batch`; unknown batch → `unknown`; order VC-10482 → delivered; recall replacement → eligible for human assessment; missing order → `needs_order`. These are real Toolbox preflight results, not native agent evaluation scores or proof of the hosted customer journey.

From the **demo repository root**, the durable verification command is:

```
.\agent\src\.venv\Scripts\python.exe .\scripts\probe_toolbox.py --project-endpoint
https://fdy-volt-citadel-261111.services.ai.azure.com/api/projects/volt-dev --version 2 --out
.\.runtime\volt-tool-retrieval.json
```

The saved evidence is `.\.runtime\volt-tool-retrieval.json`, relative to the repository root. This command is for preflight, not an instruction to rerun it during document preparation. SDK/CLI setup requires the actual azd-managed `FOUNDRY_PROJECT_ENDPOINT` binding.

Direct-tool preflight, DEV baseline/candidate grading, pinned DEV v5 Memory, Optimizer outcome, DEMO post-promotion grading and fresh DEMO observability are separate proofs. Cross-user Memory isolation and human timing rehearsal remain undone. The Memory command is bound from README; an APIM restricted-consumer denial capture is still needed if that optional live drill is retained.

Baseline incident evidence: DEV v1/Responses 2.0.0/Toolbox 2 native response `caresp_02ddb9a77175f58700lAjFhWXx30P0Frxo1oLxhgKwmjshXgd7` contains the real four-tool sequence. Reported correlation ID `8be3ccdd0e7b0bdcac285dcbbcdf4ef4` was not found in exported App Insights. Native capture plus failed native evaluation are the verified incident evidence. Client elapsed 25.372s is one observation, not an SLA.

The baseline answer's prose and citations are right, but its model decision is `recommendation` while the eligibility tool returned `eligible`. This remains the historical incident and native v1 failure. The later v6 capture passes this business case; do not portray the baseline failure as unchanged v6 behavior or infer DEMO release from the candidate result.

Completed native baseline: business_contract v1 — **8 passed, 1 failed, 0 errored of 9**. Failed item 1 is `replacement-recall`. This new hosted **8/9** baseline is not the old local **7/9**. It grades captured real hosted responses and observed successful tool calls, not coherence and not a server-side rerun. Replay artifact: `recordings\baseline-v1.json`, commit `78b54e4`.

Native report: https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/evaluations/eval_5a3774acf4f347e1bd35ae30375d0f2d/run/evalrun_28da2cdf28224eb799a9a2e3d9f8672e

Completed candidate v6: 9 passed, 0 failed, 0 errored, each of nine rows verified passed. Same pinned evaluator v1; unchanged nine-case dataset. One initial model-service failed invocation was preserved and retried once. This is business grading of completed real responses, **not 100% operational availability**, and native evaluation does not rerun the agent server-side.

Candidate report: https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/evaluations/eval_5a3774acf4f347e1bd35ae30375d0f2d/run/evalrun_d6967759ba834c51b26a39296474ff0f

Candidate/release provenance: reviewed head `1d01eaae455ac0b2aa39da0f5a57d5543017a5e2`; saved ZIP SHA-256 `ac015b257905e6dbd92f04bc49d6f86c8eb757f22557433c7c2ad47a83508996`. Issue https://github.com/SridharArrabelly/foundry-showcase/issues/1 ; MERGED PR https://github.com/SridharArrabelly/foundry-showcase/pull/2 ; merge `04a2197` into `feature/volt-citadel`, main untouched. User approved merge/promotion. Same saved ZIP uploaded without rebuild to DEMO agent `volt-support` v1 and native hash verified. Local Copilot work—not failed-before-start cloud dispatch—produced the repair.

Operator gate, not GitHub policy enforcement: `scripts\promote_demo.py` checks the merged exact PR, GitHub CI, native case counts and expectations, grader hash and source hash, then uploads the same ZIP. The documented SDK/CLI multipart code route was used because `azd --from-package` lacks code-zip metadata here. Do not describe this as GitHub branch protection or protected-environment enforcement.

Finalized receipt / operator contingency: the actual `.runtime\volt-demo-release.json` now has **status=runtime-verified**. Static verifier arguments and matching local evaluation receipt are bound below. The verifier reads the completed live DEMO evaluation, checks target code hash/source lineage/configuration and pinned native grader, then writes the promotion receipt. It does not run the agent again or replace the nine-case gate with a single-case demo. This is the documented completed operation, not an instruction to rerun it on stage. Only an authorized operator should use it for an approved verification/contingency; never hand-edit the receipt to bypass a failed check.

```
$python = ".\agent\src\.venv\Scripts\python.exe"
$source = ".\agent\src"
& $python .\scripts\verify_demo_release.py `
--receipt .\.runtime\volt-demo-release.json `
--runtime-evaluation "$source\.foundry\results\demo-v1\evaluation.json"
```

Final DEMO proof: volt-demo/volt-support v1, same ZIP/native hash, nine fresh runtime cases pass. Its own native evaluator v1 completed **9 passed, 0 failed, 0 errored**, separate from DEV. Export `recordings\demo-v1.json`. Report: https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-demo/build/evaluations/eval_a0c23c81fa62456fabd5f05a11453cec/run/evalrun_c61cce9dc8774d528841ca069d992255 . DEMO's native guardrail also returned HTTP 400 / content_filter. Memory off, project-only runtime role, no Memory account roles.

Separate fresh DEMO observability proof: response `caresp_03f2cedafe9a81ff00ml4CEH7VZPqFrxR1SFFHpglDzRb3u45a`; App Insights root operation `7f4619cba4075a99c6d1b077dad68f69`; UTC `2026-11-11T17:19:09.3446407Z`; duration **23.4094s**, one observation; project volt-demo, version 1. Evidence `.runtime\volt-demo-golden-trace.json`. Correlate with `customDimensions['azure.ai.agentserver.response_id']`; use `customDimensions['gen_ai.agent.version']` and `customDimensions['microsoft.foundry.project.id']`. `gen_ai.agent.id` is a principal GUID, not name:version. This does not verify the old incident span or automated Insights monitoring/findings.

Native Optimizer outcome: initial job `opt_69c51eb68b4a43bc8c9d21b748f190c0` failed during candidate evaluation; APIM logged real upstream 429s followed by Citadel circuit-breaker 503s, not merely a local watcher/DNS failure. Mini capacity was 10 RPM / 10,000 TPM, below the 20,000-TPM

application contract. User-approved capacity 10→50 read back **50 RPM / 50,000 TPM**, same DataZoneStandard/model version/RAI policy; APIM stayed **20k TPM / 500k daily**.

One user-authorized retry, `opt_bcc5008122ae4a88ab10bfa428ecf616`, ended **succeeded** with termination **candidate_screening_stalled**. Baseline/best task-adherence **0.75/0.75**; **completed_candidate_count=0**; only baseline returned; **no accepted improvement, NONE applied**. Budget **max 2**, not two completed candidates. Scope was instruction-only, unchanged model/tool permissions, separate training/validation, no automatic application.

Retry report: https://ai.azure.com/nextgen/r/QLaHyjG1Q3S-okwXg2m5wA,rg-volt-citadel,,fdy-volt-citadel-261111,volt-dev/build/agents/volt-support/optimization/opt_bcc5008122ae4a88ab10bfa428ecf616 . Full prepared result: `.runtime\volt-optimizer-retry-result.txt`. Reported tokens: agent **143,266**, evaluation **49,373**, optimization **14,979**. Agent average **20.682 seconds across 20 calls**; not an SLA, cost guarantee or automatic APIM-budget accounting. This native-job retry is distinct from the invocation retry in the DEV business captures. Retain the Optimizer baseline; the business defect was repaired separately through the PR.

Verified guardrail: `probe_guardrail.py` produced HTTP 400, code `content_filter`, type `content_safety_error`, input stage. The recorded local evidence is `.\.runtime\volt-guardrail-proof.json`; request ID `088298eff265d56cd2438f8dbd2dcbb2`; `native_filter_observed=true`. The exact rehearsal command and the named-agent/version caveat are in slide 11's notes. This proves the configured harmless-marker block, not universal safety.

Memory single-caller lifecycle PASSED: four distinct sessions/conversations verified native remember, exact stored-content read-back, new-conversation recall, forget by native item identity, native deletion and a fourth conversation reporting no saved preference. Private evidence locator: `.runtime\memory-proof-v5-pinned\summary.json` plus four step records. These files are **not copied into the materials**. Redact native Memory IDs, caller scope and raw session IDs everywhere; no actual values are recorded here.

Pinning correction: SDK/CLI use `extra_body.agent_session_id`; native get_session and matching session identity confirmed v5 while preview agent_reference misleadingly reported DEV-current v6. The bound README command is `probe_memory.py --project-endpoint <verified DEV project endpoint> --expected-version 5 --out .\.runtime\memory-rehearsal`, with its complete invocation in slide 12. No second-identity proof or clean preview-version-metadata claim is made.

Permission and isolation limits: the documented Foundry User role for the DEV project's managed identity was required at Foundry account scope; OpenAI User alone was insufficient. A second genuine authenticated identity is still required to prove scope/user-ID binding and cross-user isolation. The shared-account DEV/DEMO profile remains **not a hard production isolation boundary**. Memory/evaluation model traffic is not automatically under the customer APIM token budget.

Issue #1 is closed as completed with the DEMO-report comment. Merged-branch CI run 43065816204 succeeded. Automated Insights monitoring/findings are **not configured**; full human 90-minute timing rehearsal and second-identity Memory proof are **not done**. Recorded evidence.html is the offline fallback, not a portal screenshot. Repository cleanup did not change Azure resources or expose private Memory records.

Appendix D — Source framing and references

The supplied workshop is the primary narrative and design source: two environments/two loops/two bridges (15); hosted as the control path (31); governed capabilities (47, 57); supported intervention boundaries (128); trust by evidence rather than a lucky run (83); evaluate/apply/release as separate decisions (98, 101); human-reviewed trace flywheel (185); gated outer loop (159); and evidence-led takeaways (198). All original finance, Nova, library and Factory narration has been replaced. The complete source-slide mapping and readiness snapshot are in `Source mapping and readiness.md`.

Official references for presenter preparation, not claims that the current runtime implements every documented feature:

- https://learn.microsoft.com/agent-framework/hosting/foundry-hosted-agent
- https://learn.microsoft.com/azure/foundry/agents/how-to/tools/toolbox
- https://learn.microsoft.com/azure/foundry/agents/concepts/what-is-foundry-iq
- https://learn.microsoft.com/azure/foundry/agents/concepts/what-is-memory
- https://learn.microsoft.com/azure/foundry/agents/concepts/hosted-agent-guardrails
- https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview
- https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/custom-evaluators
- https://learn.microsoft.com/azure/foundry/how-to/develop/cloud-evaluation
- https://github.com/Azure-Samples/foundry-citadel-platform
- https://github.com/Azure-Samples/ai-hub-gateway-solution-accelerator